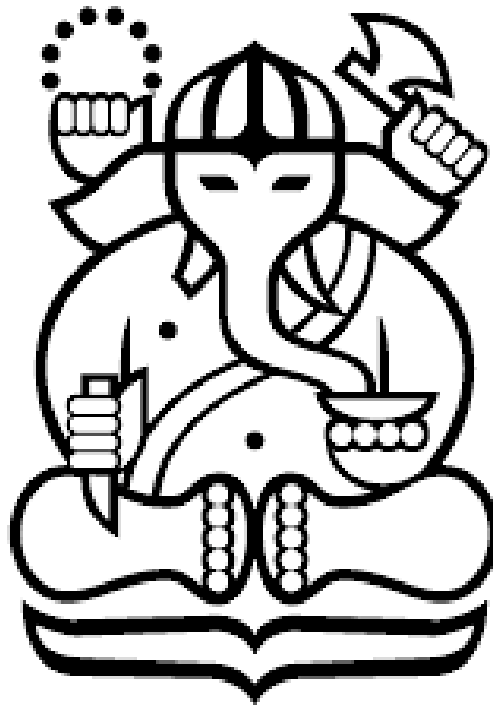


Laporan Tugas Kecil 3

IF2211 Strategi Algoritma

Ice Sliding Puzzle Solver



Disusun Oleh :

Daniel Anindito Nugroho - 13524002

Timothy Bernard Soeharto - 13524092

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Semester II 2025/2026**

A. Struktur Data yang Digunakan

1. Board

Board digunakan untuk menyimpan representasi papan permainan. Di dalam struktur ini disimpan ukuran papan, grid tile, grid cost, posisi awal aktor, posisi tujuan, serta daftar checkpoint angka yang harus dilewati secara berurutan.

Grid tile berisi simbol seperti *, X, L, Z, O, dan angka 0..9. Grid cost berisi biaya traversal setiap tile. Posisi awal aktor diperoleh dari simbol Z, sedangkan posisi tujuan diperoleh dari simbol O. Checkpoint adalah tile angka pada papan. Karena aturan permainan mewajibkan aktor melewati angka secara berurutan, checkpoint disimpan dalam urutan naik, misalnya 0, 1, 2, dan seterusnya.

2. Position

Position merepresentasikan koordinat suatu tile pada papan. Position menyimpan dua nilai utama, yaitu row dan col. Struktur ini digunakan untuk menyatakan posisi aktor, posisi goal, posisi checkpoint, serta tile-tile yang dilewati saat sliding. Position juga digunakan dalam perhitungan jarak Manhattan untuk heuristic.

3. State

State merepresentasikan kondisi permainan pada suatu titik pencarian. State tidak cukup hanya menyimpan posisi aktor, karena posisi yang sama dapat memiliki kondisi permainan yang berbeda tergantung checkpoint mana saja yang sudah dilewati. Oleh karena itu, State terdiri dari dua komponen utama:

- posisi aktor
- nextCheckpointIndex

Nilai nextCheckpointIndex menunjukkan checkpoint berikutnya yang harus dilewati. Misalnya, jika nextCheckpointIndex bernilai 0, maka checkpoint berikutnya adalah angka pertama yang harus dilewati. Jika sudah melewati checkpoint tersebut, nilainya bertambah menjadi 1. Dengan struktur ini, dua state dengan posisi aktor yang sama tetapi progress checkpoint berbeda akan dianggap sebagai state yang berbeda.

4. MoveResult

MoveResult menyimpan hasil dari satu gerakan sliding yang valid. Data yang disimpan meliputi arah gerakan, daftar tile yang dilewati, tile tempat aktor berhenti, cost gerakan, dan state hasil setelah gerakan tersebut dilakukan. MoveResult diperlukan karena pada permainan ini satu aksi tidak selalu berarti berpindah satu tile. Satu aksi dapat membuat aktor melewati banyak tile sampai berhenti sebelum batu.

5. Priority Queue

Priority queue digunakan untuk menyimpan frontier, yaitu kumpulan state yang sudah ditemukan tetapi belum diekspansi. Priority queue dipilih karena UCS, GBFS, dan A* selalu membutuhkan state dengan nilai prioritas terkecil. Pada UCS, prioritas ditentukan oleh $g(n)$. Pada GBFS, prioritas ditentukan oleh $h(n)$. Pada A*, prioritas ditentukan oleh $f(n) = g(n) + h(n)$.

6. HashMap Best Cost

HashMap digunakan untuk menyimpan cost terbaik yang pernah ditemukan untuk setiap state. Struktur ini mencegah program mengeksplorasi ulang state yang sama apabila state tersebut sudah pernah dicapai dengan cost yang lebih rendah. Key dari HashMap adalah State, sedangkan value-nya adalah cost terbaik menuju state tersebut. Contohnya, jika suatu state pernah dicapai dengan cost 20, lalu ditemukan lagi state yang sama dengan cost 35, maka jalur baru tersebut dapat diabaikan karena tidak mungkin lebih baik.

State yang digunakan sebagai key harus mencakup posisi aktor dan nextCheckpointIndex. Hal ini penting karena posisi yang sama belum tentu berarti kondisi permainan sama. Posisi yang sama sebelum melewati checkpoint 0 berbeda dengan posisi yang sama setelah melewati checkpoint 0.

7. SearchResult

SearchResult digunakan untuk menyimpan hasil akhir pencarian. Struktur ini berisi informasi apakah solusi ditemukan, urutan gerakan solusi, total cost, jumlah iterasi, waktu eksekusi, daftar state pada solusi, dan daftar state yang dieksplorasi.

Struktur ini menjadi penghubung antara bagian solver dan bagian antarmuka pengguna. Solver hanya bertugas mencari solusi, sedangkan bagian CLI atau GUI dapat menggunakan SearchResult untuk menampilkan solusi, melakukan playback, atau menyimpan hasil ke file.

B. Pemodelan Masalah sebagai Graf

Ice Sliding Puzzle dimodelkan sebagai graf berbobot. Setiap node pada graf adalah sebuah state permainan. Setiap edge adalah satu gerakan sliding valid ke arah atas, bawah, kiri, atau kanan. Bobot edge adalah cost dari gerakan sliding tersebut. Cost dihitung dari jumlah cost seluruh tile yang dilewati selama gerakan, tidak termasuk tile awal, tetapi termasuk tile tempat aktor berhenti.

Masalah ini lebih tepat dianggap sebagai graf daripada tree karena state yang sama dapat dicapai melalui beberapa jalur berbeda. Misalnya, aktor dapat kembali ke posisi yang sama setelah beberapa gerakan. Jika checkpoint progress juga sama, maka state tersebut benar-benar sama dan tidak perlu dieksplorasi ulang jika cost-nya lebih buruk. Dengan pemodelan graf, program dapat menghindari loop dan eksplorasi berulang menggunakan struktur bestCostByState.

C. Aturan Pembentukan Neighbor

Untuk membentuk neighbor dari suatu state, program mencoba empat arah gerakan: atas, bawah, kiri, dan kanan. Pada setiap arah, aktor disimulasikan bergerak terus sampai salah satu kondisi terjadi. Jika aktor menabrak batu X, maka aktor berhenti tepat sebelum batu tersebut. Jika aktor keluar dari batas papan, gerakan dianggap tidak valid. Jika aktor melewati lava L, gerakan dianggap tidak valid. Jika aktor melewati checkpoint angka yang belum waktunya, gerakan juga tidak valid.

Jika aktor melewati checkpoint angka yang sesuai urutan, maka nextCheckpointIndex dinaikkan. Setelah suatu checkpoint dilewati, tile angka tersebut dapat dilewati kembali seperti tile biasa. Goal hanya dianggap valid jika aktor berhenti tepat pada tile O dan semua checkpoint sudah dilewati.

D. Algoritma UCS, GBFS, dan A*

1. Definisi $g(n)$, $h(n)$, dan $f(n)$

Pada algoritma pathfinding, digunakan beberapa fungsi utama:

- $g(n)$ adalah cost aktual dari state awal sampai state n . Nilai ini bukan estimasi, melainkan cost yang benar-benar sudah dikeluarkan.
- $h(n)$ adalah estimasi cost dari state n menuju target berikutnya atau goal. Nilai ini diperoleh dari heuristic.
- $f(n)$ adalah nilai prioritas yang digunakan oleh algoritma untuk menentukan state mana yang akan diekspansi terlebih dahulu.

Pada UCS:

$$f(n) = g(n)$$

Pada GBFS:

$$f(n) = h(n)$$

Pada A*:

$$f(n) = g(n) + h(n)$$

2. Uniform Cost Search

Uniform Cost Search selalu memilih state dengan cost aktual terkecil dari start. Algoritma ini tidak menggunakan heuristic.

UCS cocok digunakan pada graf berbobot karena mempertimbangkan total cost yang benar-benar sudah dikeluarkan. Selama semua cost bernilai non-negatif, UCS menjamin solusi optimal.

Pseudocode UCS:

```
Mulai
Buat priority queue frontier
Masukkan state awal dengan cost 0
Simpan cost terbaik state awal = 0

Selama frontier tidak kosong:
  Ambil node dengan  $g(n)$  terkecil dari frontier

Jika node adalah goal:
  Rekonstruksi jalur solusi
  Kembalikan hasil solusi
```

```
Generate semua neighbor valid dari node

Untuk setiap neighbor:
    Hitung newCost = g(current) + cost gerakan

    Jika neighbor belum pernah dicapai
    atau newCost lebih kecil dari cost terbaik sebelumnya:
        Simpan newCost sebagai cost terbaik neighbor
        Simpan parent neighbor
        Masukkan neighbor ke frontier
Jika frontier kosong:
    Kembalikan hasil tidak ditemukan
Selesai
```

Penjelasan UCS:

UCS mengeksplorasi jalur berdasarkan total cost terkecil. Algoritma ini tidak peduli apakah suatu state terlihat dekat dengan goal atau tidak. Selama cost menuju state tersebut paling kecil, state tersebut akan diproses terlebih dahulu. Karena itu, UCS dapat menemukan solusi optimal, tetapi sering mengeksplorasi banyak state.

3. Greedy Best First Search

Greedy Best First Search memilih state berdasarkan heuristic terkecil. Algoritma ini mencoba bergerak ke state yang terlihat paling dekat dengan target.

GBFS tidak menggunakan $g(n)$ sebagai prioritas utama. Oleh karena itu, GBFS dapat lebih cepat menemukan solusi, tetapi tidak menjamin solusi dengan cost minimum.

Pseudocode GBFS:

```
Mulai
Buat priority queue frontier
Hitung h(state awal)
Masukkan state awal ke frontier
Simpan cost terbaik state awal = 0

Selama frontier tidak kosong:
    Ambil node dengan h(n) terkecil dari frontier

    Jika node adalah goal:
        Rekonstruksi jalur solusi
        Kembalikan hasil solusi
```

```
Generate semua neighbor valid dari node
```

```
Untuk setiap neighbor:
```

```
    Hitung newCost = g(current) + cost gerakan
```

```
    Hitung h(neighbor)
```

```
    Jika neighbor belum pernah dicapai
```

```
    atau newCost lebih kecil dari cost terbaik sebelumnya:
```

```
        Simpan newCost sebagai cost terbaik neighbor
```

```
        Simpan parent neighbor
```

```
        Masukkan neighbor ke frontier dengan prioritas h(neighbor)
```

```
Jika frontier kosong:
```

```
Kembalikan hasil tidak ditemukan
```

```
Selesai
```

Penjelasan GBFS:

GBFS lebih agresif dalam menuju target karena hanya melihat nilai heuristic. Pada beberapa kasus, hal ini membuat GBFS menemukan solusi dengan iterasi lebih sedikit. Namun, karena tidak memprioritaskan cost aktual, GBFS dapat memilih jalur yang terlihat dekat tetapi memiliki cost besar.

4. A* Search

A* menggabungkan UCS dan GBFS. Algoritma ini mempertimbangkan cost aktual yang sudah dikeluarkan dan estimasi cost tersisa.

A* menggunakan fungsi:

$$f(n) = g(n) + h(n)$$

Jika heuristic yang digunakan admissible, A* menjamin solusi optimal.

Pseudocode A*:

```
Mulai
```

```
Buat priority queue frontier
```

```
Hitung h(state awal)
```

```
Masukkan state awal ke frontier dengan prioritas g(n) + h(n)
```

```
Simpan cost terbaik state awal = 0
```

```
Selama frontier tidak kosong:
```

```
    Ambil node dengan f(n) terkecil dari frontier
```

```
    Jika node adalah goal:
```

```
Rekonstruksi jalur solusi  
Kembalikan hasil solusi
```

```
Generate semua neighbor valid dari node
```

```
Untuk setiap neighbor:
```

```
    Hitung  $g(\text{neighbor}) = g(\text{current}) + \text{cost gerakan}$ 
```

```
    Hitung  $h(\text{neighbor})$ 
```

```
    Hitung  $f(\text{neighbor}) = g(\text{neighbor}) + h(\text{neighbor})$ 
```

```
    Jika neighbor belum pernah dicapai
```

```
    atau  $g(\text{neighbor})$  lebih kecil dari cost terbaik sebelumnya:
```

```
        Simpan  $g(\text{neighbor})$  sebagai cost terbaik neighbor
```

```
        Simpan parent neighbor
```

```
        Masukkan neighbor ke frontier dengan prioritas  $f(\text{neighbor})$ 
```

```
Jika frontier kosong:
```

```
Kembalikan hasil tidak ditemukan
```

```
Selesai
```

Penjelasan A*:

A* lebih seimbang dibanding UCS dan GBFS. Bagian $g(n)$ membuat algoritma tetap memperhatikan cost aktual, sedangkan $h(n)$ membantu mengarahkan pencarian menuju target. Dengan heuristic yang baik, A* dapat mengeksplorasi lebih sedikit state dibanding UCS, tetapi tetap mempertahankan optimalitas.

E. Heuristic yang Digunakan

1. H1: Manhattan Next Target

H1 menghitung jarak Manhattan dari posisi aktor saat ini ke target berikutnya. Jika masih ada checkpoint angka yang belum dilewati, target berikutnya adalah checkpoint tersebut. Jika semua checkpoint sudah dilewati, target berikutnya adalah goal.

Rumus H1 adalah:

$h(n) = \text{Manhattan}(\text{posisi sekarang, target berikutnya}) * \text{minimum tile cost}$

Jarak Manhattan dihitung sebagai selisih absolut baris ditambah selisih absolut kolom. H1 sederhana dan cepat dihitung, tetapi hanya memperhatikan target berikutnya.

2. H2: Manhattan Ordered Chain

H2 menghitung total jarak Manhattan dari posisi sekarang menuju seluruh checkpoint tersisa secara berurutan, lalu menuju goal.

Rumus H2 adalah:

$h(n) = \text{total Manhattan}(\text{posisi sekarang} \rightarrow \text{checkpoint berikutnya} \rightarrow \dots \rightarrow \text{goal}) * \text{minimum tile cost}$

H2 lebih informatif daripada H1 karena memperhitungkan seluruh urutan checkpoint yang masih harus dilewati. Dengan demikian, H2 biasanya dapat membantu A* mengeksplorasi state lebih sedikit dibandingkan H1.

3. H3: Chebyshev Next Target

H3 menghitung jarak Chebyshev dari posisi aktor saat ini ke target berikutnya.

Rumus H3 adalah:

$$h(n) = \max(|\text{rowA} - \text{rowB}|, |\text{colA} - \text{colB}|) * \text{minimum tile cost}$$

Heuristic ini biasanya menghasilkan nilai yang lebih kecil daripada Manhattan. Karena lebih kecil, H3 cenderung lebih aman sebagai estimasi bawah, tetapi biasanya kurang informatif.

4. H4: Euclidean Next Target

H4 menghitung jarak garis lurus dari posisi aktor saat ini ke target berikutnya menggunakan jarak Euclidean.

Rumus H4 adalah:

$$h(n) = \text{floor}(\sqrt{(\text{rowA} - \text{rowB})^2 + (\text{colA} - \text{colB})^2}) * \text{minimum tile cost}$$

Nilai Euclidean dibulatkan ke bawah agar estimasi tidak melebihi-lebihkan cost sebenarnya. H4 juga biasanya lebih kecil daripada Manhattan, sehingga tetap aman digunakan sebagai heuristic A*.

F. Admissibility Heuristic pada A*

Heuristic disebut admissible jika tidak pernah melebihi-lebihkan cost sebenarnya dari suatu state menuju goal. Dengan kata lain, untuk setiap state n , nilai $h(n)$ harus lebih kecil atau sama dengan cost optimal sebenarnya dari n menuju goal.

Keempat heuristic yang digunakan, yaitu H1, H2, H3, dan H4, bersifat admissible karena semuanya merupakan estimasi bawah terhadap cost sebenarnya.

H1 menggunakan jarak Manhattan ke target berikutnya. Jarak Manhattan mengabaikan rintangan, lava, aturan sliding, dan keharusan berhenti sebelum batu. Karena itu, nilai Manhattan tidak akan melebihi cost aktual minimum pada permainan sebenarnya.

H2 menjumlahkan jarak Manhattan menuju checkpoint tersisa secara berurutan sampai goal. Karena permainan memang mewajibkan checkpoint dilewati dalam urutan tersebut, penjumlahan ini tetap menjadi batas bawah dari lintasan valid yang harus melewati checkpoint-checkpoint tersebut.

H3 menggunakan jarak Chebyshev. Untuk dua titik pada grid, jarak Chebyshev selalu lebih kecil atau sama dengan jarak Manhattan. Oleh karena itu, estimasi H3 juga tidak akan melebihi cost aktual minimum.

H4 menggunakan jarak Euclidean yang dibulatkan ke bawah. Jarak Euclidean juga tidak lebih besar daripada jarak Manhattan pada grid. Dengan pembulatan ke bawah, H4 tetap menjadi estimasi bawah terhadap cost sebenarnya.

Semua heuristic dikalikan dengan minimum tile cost pada papan. Karena minimum tile cost adalah cost terkecil yang mungkin dikeluarkan saat melewati tile valid, perkalian ini tetap menjaga nilai heuristic sebagai batas bawah. Dengan demikian, A* dengan H1, H2, H3, maupun H4 tetap dapat menghasilkan solusi optimal, dengan asumsi semua cost tile bernilai non-negatif.

G. Perbandingan Teoritis Algoritma

UCS menggunakan prioritas $g(n)$. Algoritma ini menjamin solusi optimal karena selalu memperluas jalur dengan cost aktual terkecil. Kekurangannya adalah UCS tidak memiliki informasi arah menuju goal, sehingga dapat mengeksplorasi banyak state.

GBFS menggunakan prioritas $h(n)$. Algoritma ini biasanya dapat lebih cepat menuju goal karena menggunakan heuristic. Namun, GBFS tidak menjamin solusi optimal karena tidak mempertimbangkan cost yang sudah dikeluarkan sebagai prioritas utama.

A* menggunakan prioritas $g(n) + h(n)$. Algoritma ini menggabungkan kelebihan UCS dan GBFS. A* memperhatikan cost aktual sekaligus estimasi cost tersisa. Jika heuristic admissible, A* tetap optimal. Dalam banyak kasus, A* lebih efisien daripada UCS karena heuristic membantu mengarahkan pencarian.

Pada Ice Sliding Puzzle, cost tile dapat berbeda-beda. Karena itu, algoritma yang hanya melihat kedekatan ke goal seperti GBFS dapat memilih jalur yang terlihat baik tetapi sebenarnya mahal. UCS dan A* lebih cocok jika tujuan utama adalah menemukan solusi dengan cost minimum.

Dari sisi heuristic, H2 biasanya paling informatif karena memperhitungkan seluruh checkpoint tersisa. H1 hanya melihat target berikutnya, sehingga lebih sederhana. H3 dan H4 juga admissible, tetapi biasanya lebih lemah daripada Manhattan karena nilai Chebyshev dan Euclidean cenderung lebih kecil. Heuristic yang lebih kecil tetap aman untuk A*, tetapi dapat menyebabkan lebih banyak state dieksplorasi.

H. Kompleksitas dan Analisis Singkat

Misalkan jumlah state yang mungkin adalah V dan jumlah gerakan valid antar state adalah E . Dalam puzzle ini, satu state terdiri dari posisi aktor dan indeks checkpoint berikutnya. Jika papan berukuran N kali M dan jumlah checkpoint adalah K , maka jumlah state maksimum secara kasar adalah:

$$N \text{ dikali } M \text{ dikali } (K + 1)$$

Setiap state dapat menghasilkan maksimal empat neighbor, yaitu dari arah atas, bawah, kiri, dan kanan. Namun, setiap gerakan sliding dapat memeriksa beberapa tile sampai berhenti atau menjadi tidak valid.

Dengan priority queue, operasi pengambilan dan pemasukan node membutuhkan waktu logaritmik terhadap jumlah elemen frontier.

Kompleksitas teoritis UCS dan A* pada graph search umumnya dapat ditulis sebagai:

$$O((V + E) \log V)$$

GBFS juga menggunakan priority queue dan dapat memiliki kompleksitas serupa dalam kasus terburuk. Namun, karena GBFS tidak menjamin optimalitas, jumlah state yang dieksplorasi pada praktiknya dapat lebih sedikit atau lebih banyak tergantung kualitas heuristic dan bentuk papan.

Dari sisi memori, program menyimpan frontier, bestCostByState, parent pointer, dan exploredStates. Oleh karena itu, kebutuhan memori dalam kasus terburuk berada pada orde $O(V)$.

I. Komponen Utama Program

Komponen utama program adalah sebagai berikut.

- **Board** menyimpan representasi papan dan informasi penting seperti posisi start, posisi goal, grid tile, grid cost, dan checkpoint.
- **Movement** bertanggung jawab membentuk neighbor dari suatu state. Komponen ini menjalankan simulasi sliding berdasarkan aturan permainan.
- **Solver** bertanggung jawab menjalankan algoritma UCS, GBFS, dan A*. Solver menggunakan priority queue untuk memilih state berikutnya yang akan diproses.
- **Heuristics** bertanggung jawab menghitung nilai $h(n)$ untuk GBFS dan A*. Empat heuristic yang digunakan adalah Manhattan Next Target, Manhattan Ordered Chain, Chebyshev Next Target, dan Euclidean Next Target.
- **SearchResult** menyimpan hasil pencarian, termasuk apakah solusi ditemukan, string gerakan, total cost, jumlah iterasi, waktu eksekusi, daftar state solusi, dan daftar state yang dieksplorasi.
- **PuzzleParser** dan **ParseException** bertanggung jawab membaca file input .txt, memvalidasi format N M, membaca grid papan, membaca grid cost, memeriksa simbol valid, memastikan hanya ada satu Z dan satu O, memvalidasi cost non-negatif, serta memastikan checkpoint berurutan mulai dari 0. Jika input valid, parser membentuk objek Board.
- Komponen GUI berada pada package gui. **Main** menjadi entry point program, sedangkan PuzzleFrame menyediakan tampilan utama berupa file chooser, pilihan algoritma, pilihan heuristic, tombol solve, panel hasil, kontrol playback, dan tombol save output.
- **BoardPanel** bertanggung jawab sebagai renderer papan. Komponen ini menggambar board awal dan board pada setiap state playback. Posisi aktor terbaru digambar sebagai Z, sedangkan posisi start lama dikembalikan menjadi * jika aktor sudah bergerak.
- **ResultPanel** bertanggung jawab menampilkan ringkasan hasil pencarian, yaitu solusi gerakan, total cost, jumlah iterasi, jumlah explored states, waktu eksekusi, serta kontrol playback seperti previous, next, slider, play/pause, pilihan mode playback, dan pilihan speed.

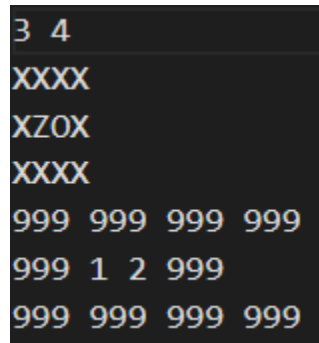
- **SolutionSaver** bertanggung jawab menyimpan hasil pencarian ke file .txt. Informasi yang disimpan meliputi path input, algoritma, heuristic, status ditemukan/tidak, move string, total cost, iterasi, waktu eksekusi, visualisasi setiap step solusi, dan visualisasi seluruh explored states.

J. Hasil Pengujian

Catatan: waktu eksekusi dapat sedikit berbeda pada setiap komputer. Pada pengujian ini beberapa testcase kecil menghasilkan waktu 0 ms karena proses selesai kurang dari 1 milidetik.

Test case 1: tc1.txt

Deskripsi : Kasus paling sederhana. Aktor berada tepat di sebelah goal dan cukup bergerak ke kanan.



Algoritma	Solusi	Cost	Iterasi	Waktu
UCS	R	2	2	0.104 ms
GBFS H1 Manhattan Next Target	R	2	2	0.047 ms
GBFS H2 Manhattan Ordered Chain	R	2	2	0.113 ms
GBFS H3 Chebyshev Next Target	R	2	2	0.104 ms
GBFS H4 Euclidean Next Target	R	2	2	0.196 ms
A* H1 Manhattan Next Target	R	2	2	0.113 ms
A* H2 Manhattan Ordered Chain	R	2	2	0.097 ms
A* H3 Chebyshev Next Target	R	2	2	0.169 ms
A* H4 Euclidean Next Target	R	2	2	0.156 ms

Test case 2: t2.txt

Deskripsi : Aktor bergerak satu arah ke kanan dan melewati checkpoint 0 sebelum mencapai goal.

```

3 6
XXXXXX
XZ*00X
XXXXXX
999 999 999 999 999 999
999 1 2 3 4 999
999 999 999 999 999 999

```

Algoritma	Solusi	Cost	Iterasi	Waktu
UCS	R	9	2	0.057 ms
GBFS H1 Manhattan Next Target	R	9	2	0.114 ms
GBFS H2 Manhattan Ordered Chain	R	9	2	0.052 ms
GBFS H3 Chebyshev Next Target	R	9	2	0.114 ms
GBFS H4 Euclidean Next Target	R	9	2	0.064 ms
A* H1 Manhattan Next Target	R	9	2	0.146 ms
A* H2 Manhattan Ordered Chain	R	9	2	0.210 ms
A* H3 Chebyshev Next Target	R	9	2	0.151 ms
A* H4 Euclidean Next Target	R	9	2	0.187 ms

Test case 3: tc3.txt

Deskripsi : Aktor harus melakukan dua gerakan, yaitu kanan lalu bawah, untuk mencapai goal.

```

5 5
XXXXX
XZ**X
X*X*X
X**0X
XXXXX
999 999 999 999 999
999 1 2 3 999
999 4 999 5 999
999 6 7 8 999
999 999 999 999 999

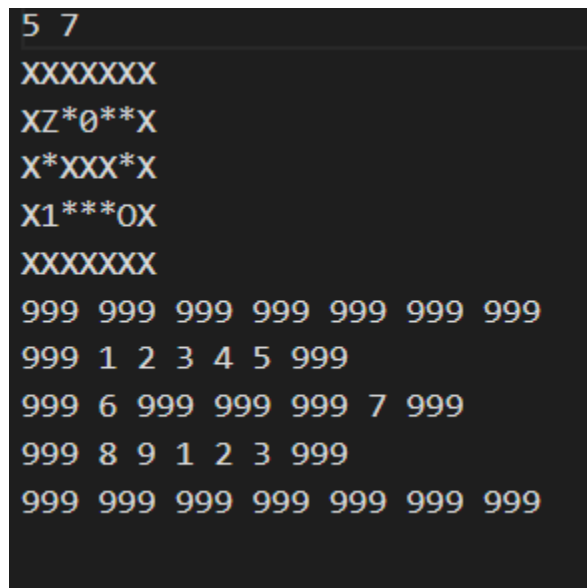
```

Algoritma	Solusi	Cost	Iterasi	Waktu
-----------	--------	------	---------	-------

UCS	RD	18	4	0.098 ms
GBFS H1 Manhattan Next Target	RD	18	3	0.139 ms
GBFS H2 Manhattan Ordered Chain	RD	18	3	0.083 ms
GBFS H3 Chebyshev Next Target	RD	18	3	0.199 ms
GBFS H4 Euclidean Next Target	RD	18	3	0.509 ms
A* H1 Manhattan Next Target	RD	18	4	0.082 ms
A* H2 Manhattan Ordered Chain	RD	18	4	0.120 ms
A* H3 Chebyshev Next Target	RD	18	4	0.104 ms
A* H4 Euclidean Next Target	RD	18	4	0.086 ms

Test case 4: tc4.txt

Deskripsi : Aktor harus melewati checkpoint 0, lalu checkpoint 1, kemudian mencapai goal.



Algoritma	Solusi	Cost	Iterasi	Waktu
UCS	RLDR	53	7	0.269 ms
GBFS H1 Manhattan Next Target	RLDR	53	6	0.095 ms
GBFS H2 Manhattan Ordered Chain	RLDR	53	5	0.172 ms
GBFS H3 Chebyshev Next Target	RLDR	53	6	0.091 ms
GBFS H4 Euclidean Next Target	RLDR	53	6	0.165 ms
A* H1 Manhattan Next Target	RLDR	53	7	0.092 ms
A* H2 Manhattan Ordered Chain	RLDR	53	7	0.266 ms
A* H3 Chebyshev Next Target	RLDR	53	7	0.551 ms
A* H4 Euclidean Next Target	RLDR	53	7	0.187 ms

Test case 5: tc5.txt

Deskripsi : Testcase 7x7 dengan checkpoint 0 dan 1, lava, wall, serta beberapa kemungkinan arah sliding.

```

7 7
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999

```

Algoritma	Solusi	Cost	Iterasi	Waktu
UCS	RULUDRUR	87	13	0.140 ms
GBFS H1 Manhattan Next Target	RULUDRUR	87	11	0.327 ms
GBFS H2 Manhattan Ordered Chain	RULUDRUR	87	9	0.119 ms
GBFS H3 Chebyshev Next Target	RULUDRUR	87	12	0.189 ms
GBFS H4 Euclidean Next Target	RULUDRUR	87	11	0.171 ms
A* H1 Manhattan Next Target	RULUDRUR	87	13	0.168 ms
A* H2 Manhattan Ordered Chain	RULUDRUR	87	13	0.242 ms
A* H3 Chebyshev Next Target	RULUDRUR	87	13	0.316 ms
A* H4 Euclidean Next Target	RULUDRUR	87	13	0.350 ms

Test Case 6 : invalid_symbol.txt

Deskripsi: Testcase invalid untuk menguji validasi simbol pada grid papan. File ini memiliki simbol A, padahal simbol yang valid hanya *, X, L, Z, O, dan digit 0 sampai 9.

```

3 4
XXXX
XZA0
XXXX
999 999 999 999
999 1 2 3
999 999 999 999

```

Hasil pengujian: program menolak input dan menampilkan pesan error Invalid board symbol 'A' at row 2, column 3.

Test Case 7 : checkpoint_gap.txt

Deskripsi: Testcase invalid untuk menguji validasi urutan checkpoint. File ini memiliki checkpoint 0 dan 2, tetapi tidak memiliki checkpoint 1, sehingga checkpoint tidak kontigu dari 0.

```
5 5
XXXXX
X0**X
XZ20X
X***X
XXXXX
999 999 999 999 999
999 1 2 3 999
999 4 5 6 999
999 7 8 9 999
999 999 999 999 999
```

Hasil pengujian: program menolak input dan menampilkan pesan error Checkpoint digits must be contiguous starting from 0.

K. Analisis Hasil Pengujian

Berdasarkan hasil pengujian, seluruh algoritma berhasil menemukan solusi yang sama pada lima testcase valid yang diuji. Cost solusi yang dihasilkan juga sama untuk semua algoritma pada masing-masing testcase. Hal ini menunjukkan bahwa pada testcase yang digunakan, ruang pencarian relatif kecil dan jalur solusi yang tersedia cukup terbatas, sehingga GBFS pun menemukan solusi dengan cost yang sama seperti UCS dan A*.

UCS menghasilkan solusi dengan cost minimum karena algoritma ini memilih state berdasarkan cost aktual dari start. Namun, UCS tidak menggunakan heuristic sehingga pada beberapa kasus jumlah iterasinya lebih banyak. Hal ini terlihat pada tc3.txt, tc4.txt, dan tc5.txt, di mana UCS mengeksplorasi lebih banyak state dibandingkan GBFS.

GBFS cenderung mengeksplorasi lebih sedikit state karena pencarian diarahkan oleh heuristic. Pada testcase tc5.txt, GBFS H2 hanya membutuhkan 9 iterasi, lebih sedikit dibandingkan GBFS H1 yang membutuhkan 11 iterasi dan UCS yang membutuhkan 13 iterasi. Ini sesuai dengan teori bahwa heuristic yang lebih informatif dapat membantu algoritma lebih cepat mendekati target.

A* pada pengujian ini menghasilkan cost yang sama dengan UCS. Namun, jumlah iterasi A* tidak selalu lebih kecil daripada UCS. Pada beberapa testcase, A* H1, H2, H3, dan H4 tetap mengeksplorasi jumlah state yang sama dengan UCS. Hal ini dapat terjadi karena ukuran papan kecil,

jumlah percabangan terbatas, dan nilai heuristic belum cukup kuat untuk mengubah urutan eksplorasi secara signifikan dibandingkan UCS.

H2 Manhattan Ordered Chain terlihat paling membantu pada GBFS untuk testcase yang memiliki checkpoint, terutama pada tc4.txt dan tc5.txt. H2 lebih informatif karena memperhitungkan jarak menuju seluruh checkpoint tersisa dan goal, bukan hanya target berikutnya. Pada A*, perbedaan H1 sampai H4 belum terlihat signifikan pada testcase ini karena semua varian menghasilkan jumlah iterasi yang sama.

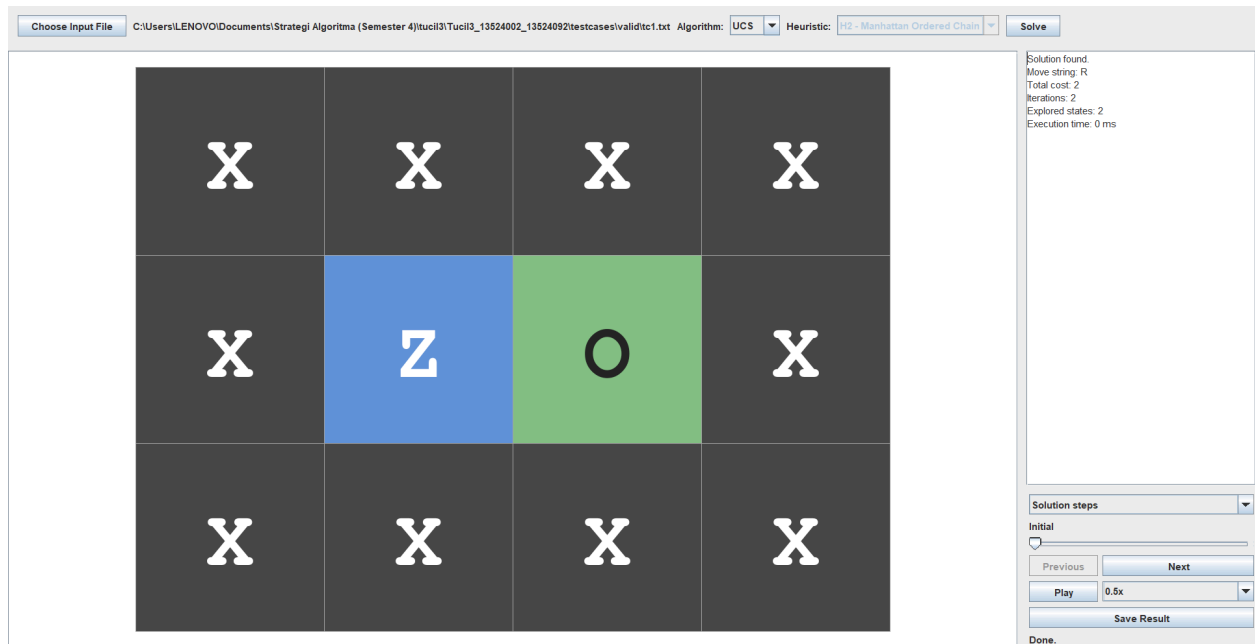
H3 Chebyshev dan H4 Euclidean tetap dapat digunakan sebagai heuristic, tetapi pada testcase yang diuji tidak memberikan pengurangan iterasi dibandingkan Manhattan. Hal ini sesuai dengan perkiraan bahwa nilai Chebyshev dan Euclidean sering lebih kecil atau kurang agresif dibandingkan Manhattan pada grid, sehingga arahan pencarian tidak selalu lebih kuat.

Secara keseluruhan, hasil aktual cukup sesuai dengan teori. UCS cocok untuk mencari solusi optimal berdasarkan cost, GBFS lebih cepat dalam jumlah iterasi pada beberapa kasus tetapi tidak menjamin optimal secara umum, dan A* menjadi pendekatan seimbang karena mempertimbangkan cost aktual serta heuristic.

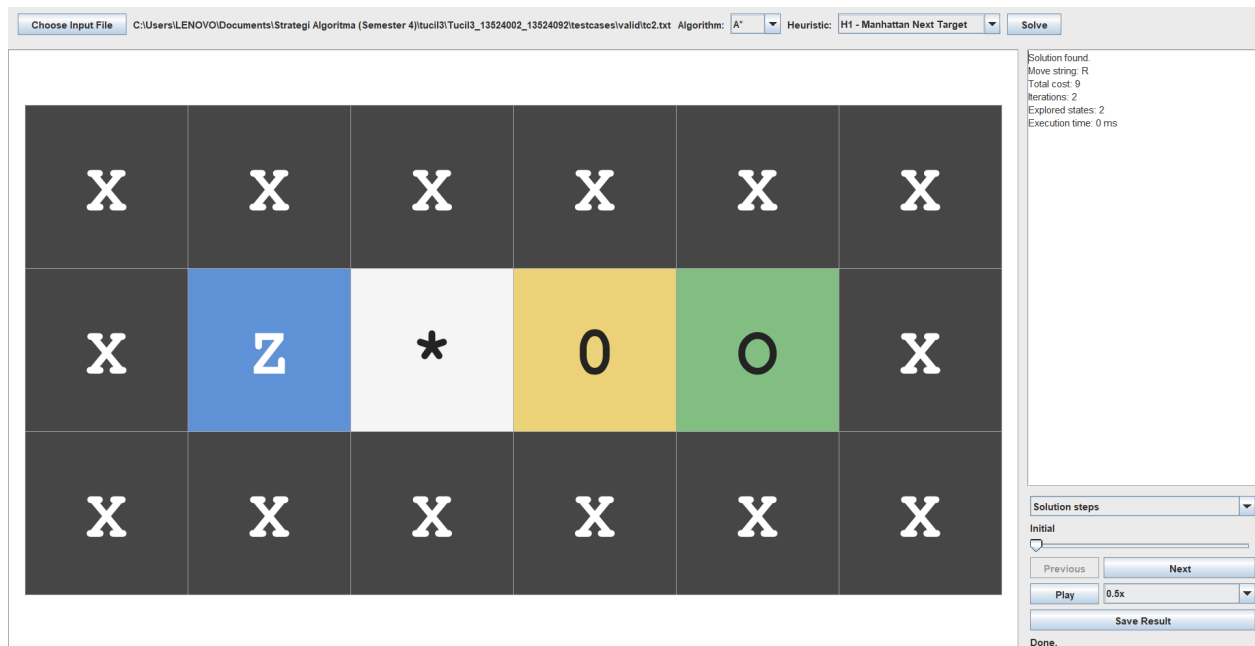
L. Tangkapan Layar

Tangkapan layar pada bagian ini diambil dari hasil menjalankan program melalui GUI. Setiap screenshot sebaiknya memperlihatkan file input yang dipilih, algoritma dan heuristic yang digunakan, visualisasi board, solusi gerakan, total cost, jumlah iterasi, dan waktu eksekusi. Gambar input .txt sudah dicantumkan pada bagian hasil pengujian sebagai pendukung deskripsi testcase.

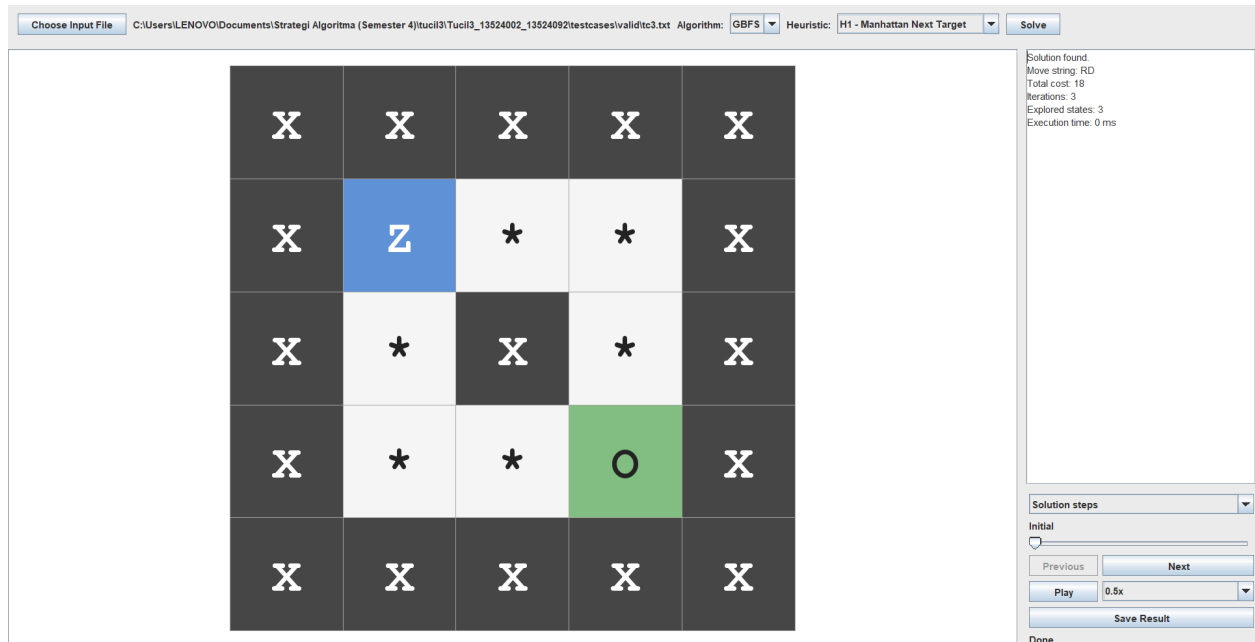
Screenshot 1: Hasil run GUI untuk tc1.txt menggunakan UCS. Tampilan menunjukkan solusi R, cost 2, jumlah iterasi 2, dan waktu eksekusi program.



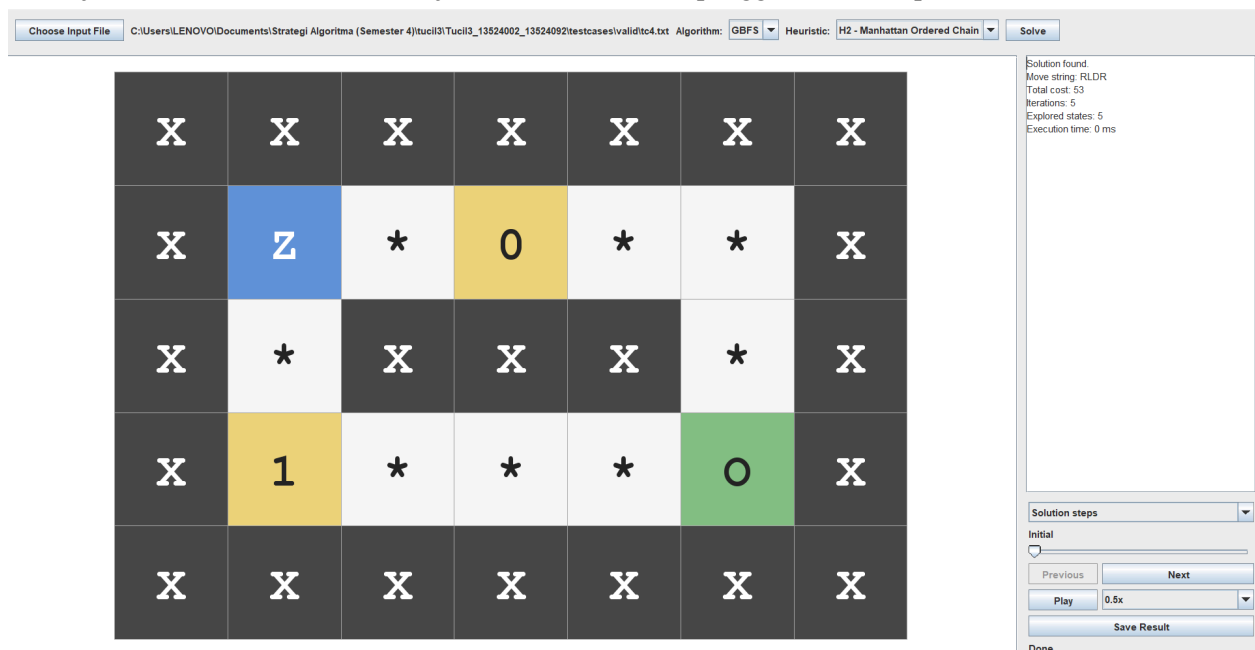
Screenshot 2: Hasil run GUI untuk tc2.txt menggunakan A* H1 Manhattan Next Target. Tampilan menunjukkan aktor melewati checkpoint 0 dan mencapai goal dengan solusi R.



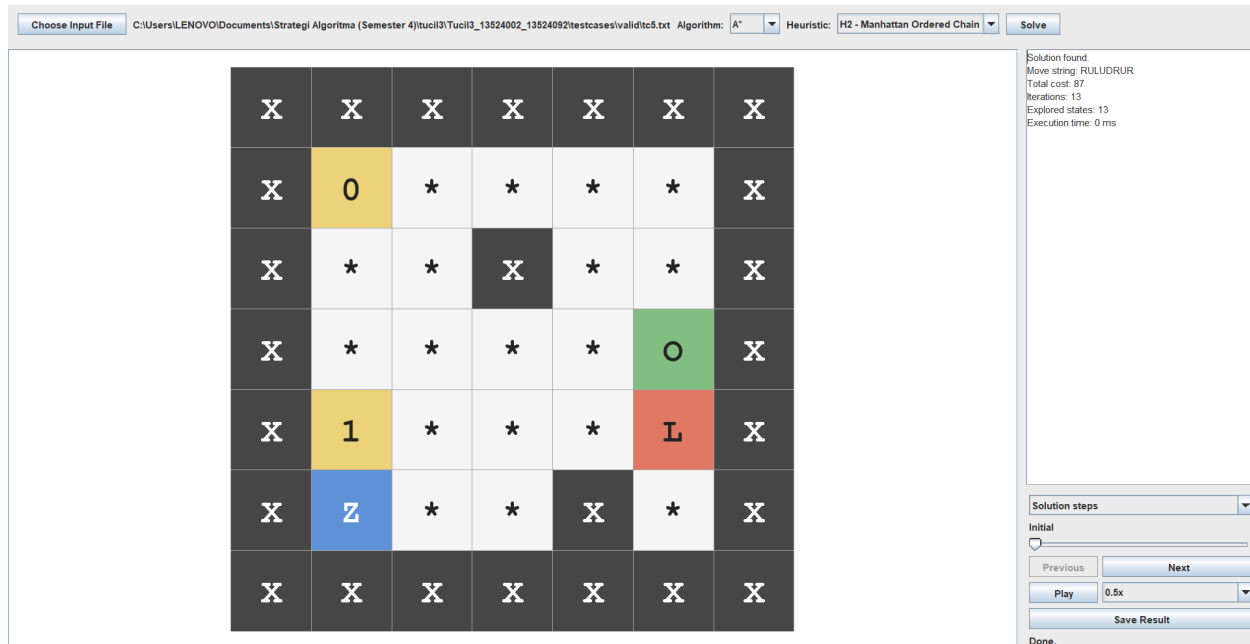
Screenshot 3: Hasil run GUI untuk tc3.txt menggunakan GBFS H1 Manhattan Next Target. Tampilan menunjukkan solusi RD, cost 18, jumlah iterasi 3, dan board pada salah satu step playback.



Screenshot 4: Hasil run GUI untuk tc4.txt menggunakan GBFS H2 Manhattan Ordered Chain. Tampilan menunjukkan solusi RLDR, cost 53, jumlah iterasi 5, serta penggunaan checkpoint 0 dan 1.



Screenshot 5: Hasil run GUI untuk tc5.txt menggunakan A* H2 Manhattan Ordered Chain. Tampilan menunjukkan solusi RULUDRUR, cost 87, jumlah iterasi 13, waktu eksekusi, dan kontrol playback.



M. Kesimpulan

Dari hasil implementasi, permainan Ice Sliding Puzzle dapat dimodelkan sebagai graf berbobot, di mana setiap node adalah state permainan dan setiap edge adalah gerakan sliding valid dengan bobot berupa cost movement.

UCS cocok digunakan ketika tujuan utama adalah memperoleh solusi optimal berdasarkan cost, tetapi algoritma ini dapat mengeksplorasi banyak state. GBFS menggunakan heuristic untuk mengarahkan pencarian, sehingga dapat lebih cepat pada beberapa kasus, tetapi tidak menjamin solusi optimal. A* menggabungkan cost aktual dan heuristic, sehingga menjadi pendekatan yang lebih seimbang.

Penggunaan state yang terdiri dari posisi aktor dan nextCheckpointIndex penting agar program dapat membedakan kondisi permainan yang terlihat sama secara posisi, tetapi berbeda dari sisi progress checkpoint. Selain itu, penggunaan HashMap untuk menyimpan cost terbaik membantu mencegah eksplorasi ulang state yang tidak lebih baik.

Berdasarkan hasil pengujian aktual, GBFS H2 Manhattan Ordered Chain menghasilkan jumlah iterasi paling sedikit pada testcase yang memiliki checkpoint. Pada tc5.txt, GBFS H2 hanya membutuhkan 9 iterasi, sedangkan UCS dan A* membutuhkan 13 iterasi. Namun, karena GBFS tidak memperhitungkan cost aktual sebagai prioritas utama, algoritma ini tetap tidak dapat dijadikan jaminan untuk solusi optimal pada semua kasus.

Pada testcase yang digunakan, semua algoritma menghasilkan solusi dengan cost yang sama. Hal ini menunjukkan bahwa testcase relatif kecil dan jalur solusi yang tersedia cukup terbatas. Untuk kasus yang lebih besar dan memiliki banyak alternatif jalur dengan cost berbeda, perbedaan antara UCS, GBFS, dan A* kemungkinan akan lebih terlihat.

Secara umum, algoritma yang paling stabil untuk memperoleh solusi optimal adalah UCS dan A*. Jika fokus utama adalah mengurangi jumlah eksplorasi pada puzzle dengan checkpoint, heuristic H2 Manhattan Ordered Chain menjadi pilihan yang paling baik pada pengujian ini.

LAMPIRAN

A. Repository Program

Link Github:

https://github.com/le1n-gif/Tucil3_13524002_13524092

B. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Daniel Anindito Nugroho - 13524002



Timothy Bernard Soeharto - 13524092

C. Checklist Penilaian

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	

5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	